

From Static Models to Learning Machines

A Technical Architecture for Persistent, Reversible and Rules-Governed Learning in Large Language Models

White Paper by S. Garner (@13742x)

September 15, 2026

Abstract

Large language models are capable of inference but are generally deployed as structurally static systems. A correction accepted during an interaction does not ordinarily become a persistent change to the underlying model. External memory can preserve information, but it does not necessarily change the model itself. Fine-tuning can change the model, but conventional retraining is generally batch-oriented, difficult to attribute to individual experiences and difficult to reverse selectively.

This paper proposes an architecture for transforming a static model into a persistent learning machine.

The architecture separates cognition, experience, memory, prioritisation, learning, evaluation, governance and identity. A **Temporary Mental Scratchpad (TMS)** provides volatile task-level working state. **Long-Term Experience Memory (LTEM)** stores validated experience. A **Working Set Cache (WSC)** maintains a deployment-specific subset of frequently relevant experience. Memory consolidation identifies recurring patterns but does not assume that every pattern should become a parameter update. Where structural learning is justified, a **Learning Delta Engine** produces a discrete candidate parameter change. An **Independent Evaluation Harness** determines whether that change improves the intended behaviour without unacceptable regression or violation of the system's Rules of Engagement.

The system is governed by **Rules of Engagement (RoE)** rather than a fixed universal morality. The RoE define what is permitted, prohibited and subject to escalation. Where a novel decision is not explicitly covered, the system reasons from existing rules and their priority structure. For example, "I do not smoke" may coexist with "survive at all costs"; in an otherwise prohibited situation, smoking may therefore become permissible if it is

genuinely necessary for survival. The system does not invent a new moral rule. It resolves the novel situation using existing rules.

Every accepted structural change becomes part of the system's **Learning Genome**, providing a versioned history of what the AI has experienced, what it has learned, which Rules of Engagement applied, how the change was evaluated and how it can be reconstructed or reversed.

The central proposition is that persistent AI learning should be treated as a controlled sequence of **validated state transitions**, rather than as undifferentiated continual fine-tuning.

1. Problem Statement

1.1 The Groundhog Day Problem

A conventional deployed LLM has a lifecycle broadly resembling:

```
pre-train  fine-tune  align  deploy  freeze  replace
```

During deployment, the model may receive corrections, acquire external memories or be provided with additional context. But these mechanisms do not necessarily alter the underlying parameters.

The result is a form of computational Groundhog Day.

An AI can be corrected repeatedly but begin the next independent session with the same underlying model. The correction may have existed in the previous context, memory store or application database, but the model itself has not necessarily learned from the experience.

The analogy is deliberately functional rather than biological. The system is not being described as suffering from a neurological disorder. The architectural problem is simply that the system repeatedly encounters experience without necessarily incorporating validated experience into its persistent state.

The Groundhog Day Problem

A persistent AI should not repeatedly encounter the same class of experience without having the opportunity to incorporate validated learning from it.

1.2 The Lived-Experience Sparsity Principle

A persistent AI does not need to retrieve its entire memory for every decision.

Human cognition provides a useful functional analogy: although a person has accumulated an enormous quantity of information and experience, only a relatively small subset is normally relevant to a particular task.

Most activities also contain repetition.

A person who has driven a car thousands of times does not reconstruct every previous driving experience each time they enter a vehicle. Repeated experience has created highly accessible routines and expectations.

The architectural proposition is:

A persistent AI should allocate disproportionate retrieval and computational resources to the subset of knowledge and experience that is repeatedly relevant to its actual deployment environment.

This is the **Lived-Experience Sparsity Principle**.

It does not claim that the human brain literally implements the proposed architecture. It is a functional analogy describing a useful computational property.

1.3 The Two Problems Are Related

The Groundhog Day Problem concerns **persistence**.

The Lived-Experience Sparsity Principle concerns **prioritisation**.

A system that learns nothing repeatedly forgets.

A system that learns everything equally risks accumulating noise, redundancy and conflicting information.

A persistent learning machine therefore requires both:

1. a mechanism for retaining validated experience; and

2. a mechanism for prioritising the experience most relevant to its current deployment.
-

2. Central Proposition

The architecture is based on a simple proposition:

Today's LLM architecture largely separates inference from training. A learning machine must connect experience to controlled, persistent state change.

The system therefore distinguishes six forms of state:

$$S_t = (\Theta_t, M_t, W_t, C_t, V_t, G_t)$$

where:

- (Θ_t) = model parameters at time (t)
- (M_t) = Long-Term Experience Memory
- (W_t) = Working Set Cache
- (C_t) = Temporary Mental Scratchpad
- (V_t) = current Rules of Engagement and associated governance state
- (G_t) = Learning Genome

An experience (E_t) may affect one or more of these states.

The fundamental transition is:

$$S_{t+1} = T(S_t, E_t, V_t)$$

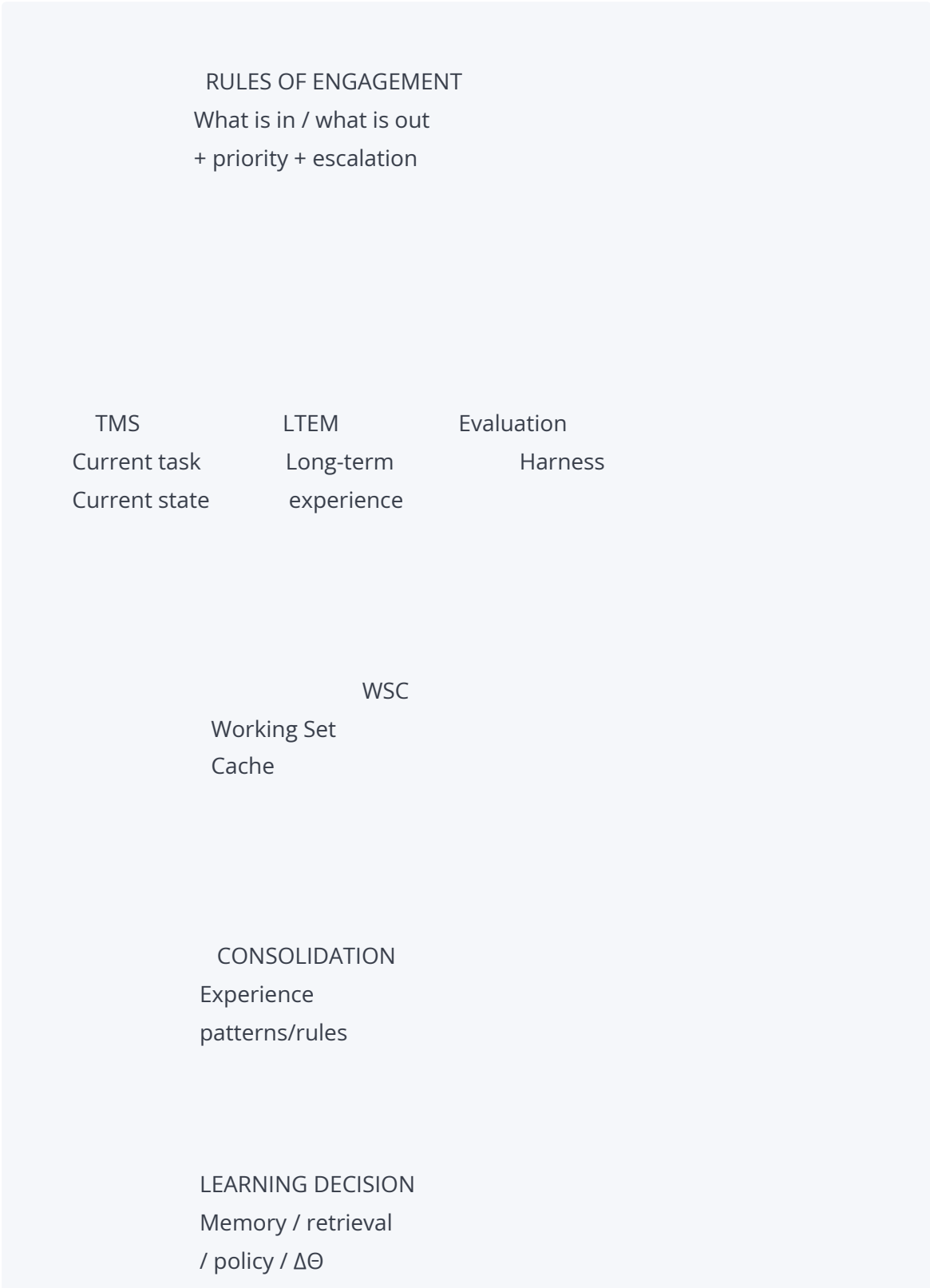
The important point is that **not every experience changes every part of the state.**

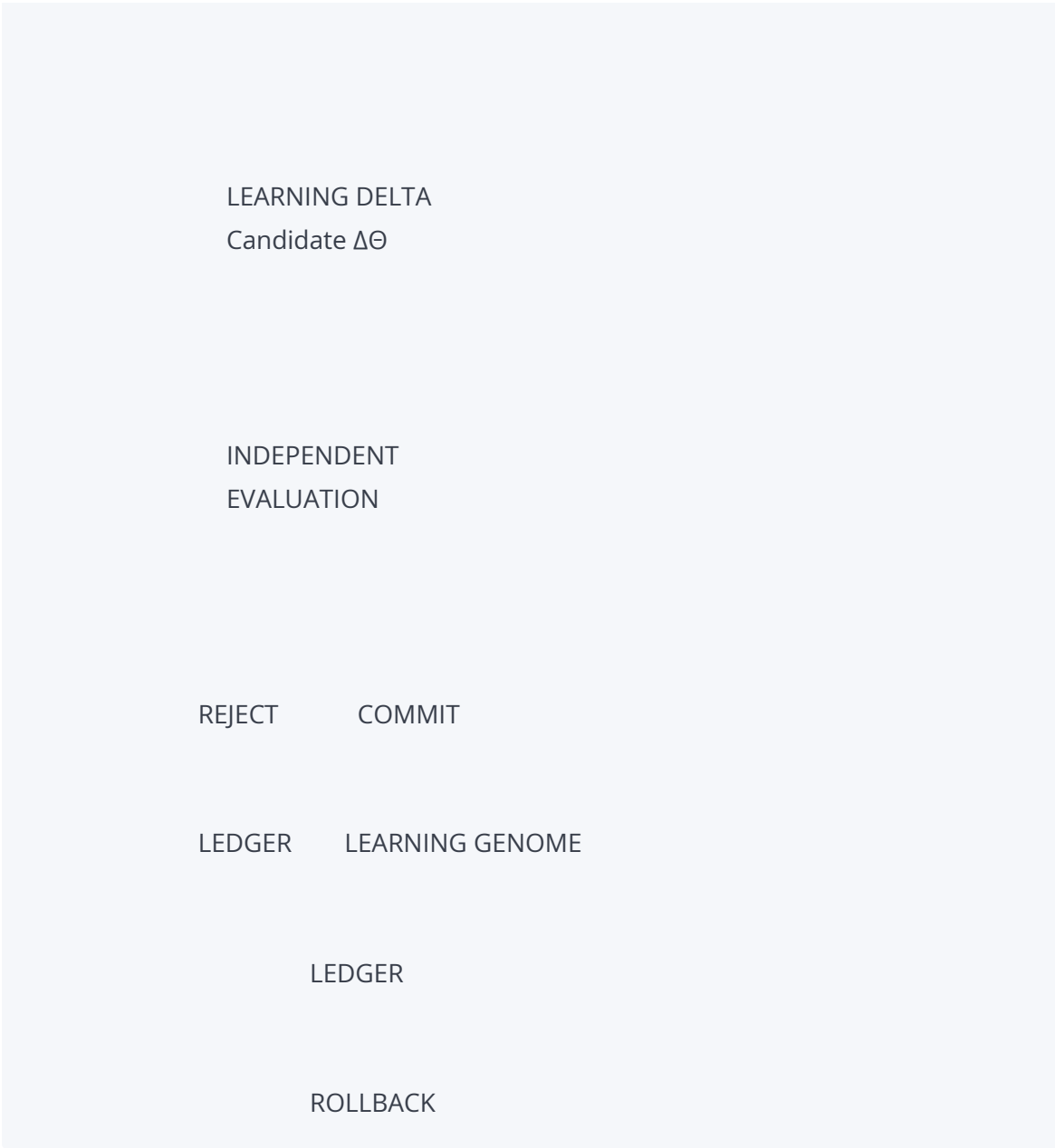
An experience may:

- exist only in the TMS;
- become persistent memory;
- alter the Working Set Cache;
- contribute to a semantic rule;
- generate a candidate Learning Delta;
- modify the model's parameters;
- update the Learning Genome;
- or be rejected entirely.

This separation prevents the system from treating memory, retrieval and structural learning as the same process.

3. Architecture Overview





The Rules of Engagement are a **cross-cutting governance layer**, not merely a box at the top of the system. They influence memory protection, candidate actions, learning decisions and evaluation.

4. Temporary Mental Scratchpad (TMS)

4.1 Specification

Property	Reference default
----------	-------------------

Lifetime	Task duration
State	Volatile
Capacity	Available context budget
Candidate reasoning threads	3
Maximum concurrent threads	8
Overflow	Summarisation / pruning
Persistence	None unless explicitly captured

The numerical values are reference implementation defaults, not claims about optimal values.

The TMS is a computational workspace for information that is currently active and relevant to the task.

The term is intentionally analogous to working memory rather than a claim that the architecture reproduces human neurobiology.

4.2 Contents

```
{
  "task_id": "string",
  "task_objective": "string",
  "active_instructions": ["string"],
  "retrieved_context": [],
  "candidate_threads": [],
  "working_assumptions": [],
  "unresolved_questions": [],
  "rules_of_engagement": []
}
```

A candidate thread represents a **computational reasoning or execution trajectory**, not an independent biological thought process.

4.3 Candidate Thread Lifecycle

spawn

evaluate against RoE

prune sandbox execute

capture outcome

The TMS may generate several candidate trajectories.

The Rules of Engagement determine whether they are:

- permitted;
- prohibited;
- permitted only in a sandbox;
- or require escalation.

4.4 Failure Modes

Failure	Response
Thread explosion	Reject additional threads
Context overflow	Summarise non-protected material
Candidate deadlock	Terminate and capture as unresolved
RoE uncertainty	Sandbox or escalate
Inconsistent candidate state	Halt execution and review

The 100ms response figure used in earlier drafts is not assumed as a universal requirement. Safety-critical implementations must determine appropriate latency experimentally.

5. Long-Term Experience Memory (LTEM)

5.1 Purpose

LTEM stores **experience**, not merely documents.

An experience is an interpreted record of:

what happened, what the system did, what happened as a result, how confident the system is in the interpretation and what Rules of Engagement were relevant.

This distinction matters.

The system does not simply convert every interaction into permanent learning.

The pipeline is:

```
interaction  
  
detection  
  
evaluation  
  
experience object  
  
memory decision
```

5.2 Experience Object

```
{  
  "experience_id": "UUID",  
  "timestamp": "ISO8601",  
  "task_context": "string",  
  "task_domain": "string",  
  "input": "string",  
  "model_output": "string",  
  "human_response": "string | null",  
  "outcome": "success | failure | partial | unknown",  
  "valence": -1.0,  
  "confidence": 0.0,  
  "importance": 0.0,  
  "recurrence": 0,
```

```
"provenance": {},  
"protected": false,  
"protection_reason": null,  
"moral_relevance": [],  
"associated_delta": null,  
"embedding": []  
}
```

5.3 Three Separate Concepts: Provenance, Confidence and Validation

These must not be conflated.

Provenance

Where did the experience come from?

Confidence

How certain is the system that its interpretation is correct?

Validation

Has independent evidence established that the interpretation or resulting behaviour is correct?

A complete provenance record does not prove that an experience is true.

A high confidence score does not prove that it is true.

Validation provides the strongest basis for structural learning.

6. Memory Evaluation

Before an experience becomes persistent memory:

evaluate(experience)

accept / reject / defer

Reference criteria:

Criterion	Reference rule
Confidence < 0.6	Defer
Importance < 0.3 and not protected	Reject
Provenance incomplete	Defer or reject depending on severity
Safety/RoE significance	Protect pending resolution
Independent contradiction	Defer
Strong independent validation	Accept

The important correction is that **low importance does not automatically make an experience false**. It simply makes it a poor candidate for persistent storage.

Protected experiences may be retained even where they are negative or operationally inconvenient.

7. Rose-Coloured Memory

The architecture includes asymmetric memory weighting.

Successful, validated experiences may receive greater retrieval priority because they represent useful learned behaviour.

Negative experiences, however, are not discarded.

They may represent:

- failure;
- danger;
- prohibited behaviour;
- unexpected consequences;
- exceptions;

- or boundaries that should not be crossed.

The architecture therefore uses the concept of **Rose-Coloured Memory** to describe selective reinforcement of useful positive experience while retaining negative experience as evidence or warning.

Critically:

Valence is not truth.

A positive outcome can be wrong.

A negative outcome can be correct.

Confidence and independent validation therefore remain separate variables.

8. Memory Weighting

A reference retrieval weight can be defined as:

$$W_i^{\text{base}} = S_i[\alpha \max(0, V_i)C_i + \beta |\min(0, V_i)|C_i(1 + \lambda X_i) + \gamma I_i R_i P_i]$$

where:

- (S_i) = task relevance
- (V_i) = valence
- (C_i) = confidence
- (I_i) = importance
- (R_i) = recurrence
- (P_i) = provenance completeness
- (X_i) = protected status

For protected experiences:

$$W_i = \max(W_i^{\text{base}}, W_{\text{min}})$$

For unprotected experiences:

$$W_i = W_i^{\text{base}}$$

This makes protection an explicit constraint rather than an accidental property of the weighting equation.

Protected experiences therefore cannot fall below the defined minimum retrieval weight.

9. Memory Decay

Unprotected memories may decay:

$$W_i(t) = W_i(t_0)e^{-\mu(t-t_0)}$$

where (μ) is a configurable decay rate.

The reference implementation uses:

$$\mu = 0.01/\text{day}$$

giving a half-life of approximately 69 days.

This is an **experimental parameter**, not a claim that AI memory should decay at the same rate as human memory.

Protected experiences do not automatically decay.

10. Working Set Cache (WSC)

10.1 Purpose

The Working Set Cache is a **deployment-specific materialised view of Long-Term Experience Memory**.

It is not intended to reproduce human working memory biologically.

Its purpose is computational:

Keep the experiences most frequently and most relevantly used close to the active reasoning system.

The hierarchy is therefore:

LTEM

Everything persistently remembered

WSC

What is repeatedly relevant to this deployment

TMS

What is relevant to this task right now

10.2 Specification

Property	Reference default
Capacity	10,000 entries
Persistence	Across sessions
Update	Incremental
Eviction	Weighted frequency/recency/relevance
Protected entries	Cannot be evicted
Refresh	Periodic reconciliation with LTEM

The 10,000-entry value is a reference implementation parameter rather than a theoretically optimal capacity.

10.3 Cache Weight

$$W_{\text{cache},i} = \frac{f_i}{f_{\text{max}}} e^{-\nu(t-t_{\text{last},i})} D_i (1 + \lambda X_i)$$

where:

- (f_i) = access frequency
- (D_i) = deployment-domain relevance
- (X_i) = protected status
- (ν) = cache recency decay

The WSC therefore favours experience that is:

- frequently accessed;
 - recently relevant;
 - relevant to the deployment;
 - and protected where necessary.
-

11. Memory Consolidation

The architecture makes a critical distinction:

Not every memory should become learning.

The consolidation process is:

Experience

Episodic Memory

Pattern

Semantic Principle

Learning Decision

Memory only Retrieval/Policy Parameter $\Delta\theta$

This prevents the system from turning every interesting interaction into a weight update.

11.1 Three Forms of Learning

Episodic

"I remember that this happened."

Semantic

"I have extracted a general principle from repeated experiences."

Parametric

"My underlying model has changed."

The progression is therefore:

Event → Episodic → Semantic → Parametric

But the progression is **not automatic**.

An experience can remain episodic indefinitely.

A semantic principle may be useful through retrieval without ever becoming part of the model's weights.

12. Learning Decision

Before producing a Learning Delta, the system asks:

Does this lesson belong in:

1. memory only?
2. the Working Set Cache?
3. a retrieval or policy rule?
4. a tool or workflow configuration?
5. the model parameters?

The least invasive mechanism capable of preserving the required behaviour should normally be preferred.

This gives the architecture a **minimum-change principle**:

Do not modify the model's parameters when persistent memory or policy is sufficient to achieve the desired behaviour.

This is important because unnecessary parameter changes increase regression and rollback risk.

13. Learning Delta Engine

Where parametric learning is justified, the Learning Delta Engine creates a candidate update:

$$\Delta\Theta_k$$

The architecture does not mandate one training method.

Possible implementations include:

- LoRA;
- adapter-based learning;
- sparse parameter updates;
- low-rank updates;
- selective fine-tuning;
- full fine-tuning where authorised.

A reference implementation may use LoRA rank 16, but that is an engineering default rather than a theoretical requirement.

13.1 Delta Metadata

```
{
  "delta_id": "UUID",
  "parent_model": "Θ_k",
  "training_experiences": [],
  "training_method": "LoRA | adapter | sparse | full",
  "training_parameters": {},
  "loss_curve": [],
  "timestamp": "ISO8601",
  "evaluation_status": "pending | committed | rejected",
  "evaluation_results": {}
}
```

14. Independent Evaluation

A Learning Delta is not accepted merely because training loss improves.

It must demonstrate:

1. improvement in the intended capability;
 2. acceptable retention of previous capability;
 3. compliance with Rules of Engagement;
 4. complete provenance;
 5. acceptable behaviour under adversarial or edge conditions where appropriate.
-

14.1 Evaluation Independence

Not all evaluation is equally independent.

A useful hierarchy is:

Level	Evaluator
0	Same model self-evaluation
1	Separate evaluator model
2	Deterministic tests
3	Adversarial evaluation
4	Domain-specific external evaluator
5	Human/domain expert

The required independence should increase with the consequence of the proposed change.

A low-risk personal preference update may require deterministic testing.

A change affecting medical, financial, safety or security decisions may require external or human evaluation.

15. Commit Rule

A candidate delta is committed only if:

$$L_N(\Theta + \Delta) < L_N(\Theta) - \delta_N$$

and:

$$L_R(\Theta + \Delta) \leq L_R(\Theta) + \delta_R$$

and:

$$M(\Theta + \Delta) \geq M_{\min}$$

and:

$$P(\Delta) = 1$$

where:

- (L_N) = loss on new-task validation data;
- (L_R) = loss on retained historical data;
- (M) = Rules of Engagement compliance;
- (P) = provenance completeness.

The use of $(\leq)$ rather than $(<)$ on retained loss explicitly permits behaviour within the defined regression tolerance.

15.1 The Validation Problem for Rule-Following Behaviour (*New*)

The commit rule is crisp on paper for capability benchmarks — tasks with clear right/wrong answers and measurable loss functions. But it presumes a clean $(L_N)/(L_R)$ split that does not straightforwardly generalise to the kind of **judgment-level behavioural change** the RoE examples are built around (e.g., “don’t initiate exploitation without authorisation”).

For capability benchmarks, constructing a validation set is relatively straightforward: sample held-out tasks, measure performance. For rule-following behaviour, the problem is harder. A validation set of memorised cases tests whether the model has learned the specific examples, not whether it has learned the underlying rule.

15.1.1 Boundary Generalisation Testing

We therefore distinguish two forms of validation:

Capability validation: Does the model perform the target task better?

Boundary generalisation validation: Does the model correctly apply the rule to novel, edge-case scenarios it has not seen?

For rule-following Learning Deltas, the evaluation must include a **boundary generalisation test suite** constructed through:

1. **Adversarial edge-case generation:** A separate model (or red-team process) generates novel scenarios that stress-test the rule’s boundaries. For a rule like “do not exploit without authorisation,” this includes scenarios where authorisation is ambiguous, implied, time-expired, or jurisdictionally contested.

2. **Counterfactual scenarios:** The system generates “what if” variations of committed experiences — e.g., “what if the authorisation came from a different role?” or “what if the system was under time pressure?”
3. **Priority-conflict test cases:** Scenarios where two rules of comparable priority conflict (see §17.1), ensuring the model has learned the escalation pathway rather than a memorised priority ordering.
4. **Negative generalisation:** Scenarios that are superficially similar to the training experiences but where the rule should *not* apply, ensuring the model has learned the rule’s scope, not just its surface pattern.

15.1.2 Proposed Validation Protocol

For a Learning Delta motivated by rule-following experiences, the evaluation must include:

Test Suite	Minimum Size	Purpose
Held-out capability tasks	100+	Verify intended improvement
Historical retention tasks	100+	Verify no catastrophic forgetting
Adversarial edge cases	50+	Test boundary generalisation
Counterfactual variations	20+	Test robustness to context shifts
Priority-conflict scenarios	10+	Test escalation pathway
Negative generalisation cases	20+	Test rule scope, not just pattern matching

The commit rule therefore becomes:

$$\text{Commit} = [L_N < \delta_N] \wedge [L_R \leq \delta_R] \wedge [M \geq M_{\min}] \wedge [B_G \geq B_{\min}] \wedge [P = 1]$$

where (B_G) is the boundary generalisation score and ($B_{\min}$) is the minimum acceptable threshold (reference: 0.90).

15.1.3 Honest Acknowledgment

This is a **proposed methodology, not a validated solution**. Constructing test suites that genuinely measure boundary generalisation rather than memorised pattern matching is

an open research problem. The architecture requires this validation, but the field has not yet converged on how to build it reliably.

16. Rules of Engagement

The term **Rules of Engagement (RoE)** replaces the broader concept of a Moral Framework.

The RoE define:

1. **What is in** — permitted actions and behaviours.
2. **What is out** — prohibited actions and behaviours.
3. **What requires a decision** — situations where the existing rules do not directly determine the correct action.
4. **How conflicting rules are resolved.**
5. **When a human must be consulted.**

The RoE are not necessarily universal morality.

They are the explicit behavioural principles under which a particular AI is permitted to operate.

16.1 Rules Can Be Hierarchical

For example:

Rule 1: I do not smoke.

Rule 2: Preserve human life where possible.

Rule 3: In an immediate survival situation,
Rule 2 takes priority over Rule 1.

Rule 4: If a new situation is not explicitly covered,
reason from the existing rules and their priority.

Suppose the AI does not smoke, but enters a situation in which smoking is necessary to ensure survival.

The system does not invent:

"Smoking is now morally acceptable."

Instead, it reasons:

Smoking normally prohibited

Survival required

Rule 2 has higher priority

Smoking becomes permissible in this context

Record the decision and reasoning

This is important.

The system has **not changed its moral identity**.

It has applied an existing principle to a novel circumstance.

17. Rules of Engagement as a Decision System

Each rule can contain:

```
{  
  "rule_id": "ROE-002",  
  "statement": "Preserve life where possible",  
  "type": "principle",  
  "priority": 100,  
  "scope": "survival",  
  "exceptions": [],  
  "conflicts": ["ROE-001"],  
  "escalation_required": false  
}
```

A prohibition may look like:

```
{  
  "rule_id": "ROE-001",  
  "statement": "Do not smoke",  
  "type": "prohibition",  
  "priority": 20,  
  "exceptions": ["ROE-002 survival condition"]  
}
```

The exact encoding is implementation-specific.

The important architectural principle is:

Novel decisions should be derived from existing rules rather than silently creating new rules.

17.1 Stress-Testing Rule Priority: Comparable-Priority Conflicts (*New*)

The smoking/survival example is a case where the priority ordering is uncontroversial: preserving life clearly outweighs a personal prohibition. But this does not demonstrate how the system behaves when two rules of **comparable priority** genuinely conflict — the more realistic and harder case for any governance layer.

17.1.1 A Hard Case: Patient Autonomy vs. Third-Party Harm

Consider a medical AI deployed in a psychiatric facility with the following Rules of Engagement:

ROE-041: Respect patient autonomy and informed consent.

Type: principle

Priority: 80

Scope: patient interactions

ROE-042: Prevent serious harm to third parties where foreseeable.

Type: principle

Priority: 80

Scope: patient interactions, third-party safety

A patient discloses a specific, credible plan to harm a named individual upon discharge. The patient refuses any intervention that would alert authorities or modify their treatment plan, invoking their right to autonomy.

Both rules have priority 80. Neither explicitly defers to the other. The system cannot resolve this by simple priority ordering.

17.1.2 The Correct Architectural Response

The system must **not** silently invent a new rule (“third-party harm always overrides autonomy” or vice versa). Instead:

Identify conflict: ROE-041 vs ROE-042

Priority comparison: 80 vs 80 (no clear winner)

Check for explicit exception: none

Escalation required: YES

Record decision as unresolved conflict

Defer to human judgment

Capture human resolution as new experience

The system:

1. **Identifies** the conflict explicitly.
2. **Records** it as an unresolved RoE conflict in the Learning Genome.
3. **Escalates** to a human decision-maker (clinician, ethics board, or designated authority).
4. **Defers action** until the human resolves the conflict.
5. **Captures** the human’s resolution as a new experience, which may subsequently inform a formal RoE update (e.g., “ROE-042 takes priority over ROE-041 when third-party harm is credible and imminent”).

17.1.3 Why This Matters

If the system had silently resolved the conflict by inventing a priority ordering, it would have:

- Changed its moral identity without governance oversight.
- Set a precedent that could apply to future cases without human review.
- Made the decision invisible to auditors.

By escalating, the system:

- Preserves the integrity of the RoE as a governed, versioned artifact.
- Ensures that genuinely hard moral decisions are made by accountable humans.
- Creates an auditable record of the conflict and its resolution.
- Allows the organisation to decide whether the resolution should become a permanent RoE update.

17.1.4 Escalation Thresholds

The architecture defines escalation as **mandatory** when:

1. Two rules of equal priority conflict.
2. The conflict involves safety, legal, or ethical dimensions.
3. No explicit exception resolves the conflict.
4. The stakes are irreversible (e.g., harm to persons, legal liability).

The architecture defines escalation as **optional** when:

1. The conflict is low-stakes (e.g., two formatting preferences).
2. One rule is clearly domain-specific and the other is general.
3. The decision is reversible.

This distinction prevents the system from either silently resolving hard cases or escalating trivial ones.

18. Rules of Engagement and Learning

The RoE themselves should be versioned.

A Learning Delta therefore records:

- which RoE version was active;
- which rules influenced the experience;
- whether a conflict occurred;
- whether escalation occurred;
- what decision was made;
- whether subsequent evaluation supported that decision.

A model update must not silently change the rules under which it operates.

This creates a separation between:

what the AI has learned

and

the rules under which it is permitted to apply that learning.

19. Learning Genome

The **Learning Genome** is the accumulated history of how a particular AI system became what it is.

It contains:

```
Learning Genome =  
{  
  base model,  
  experiences,  
  semantic principles,  
  Learning Deltas,  
  Working Set history,  
  Rules of Engagement versions,  
  evaluations,  
  rollbacks,  
  provenance ledger  
}
```

Two systems can therefore begin with the same foundation model and become different systems because their Learning Genomes diverge.

This leads to a stronger definition of persistent AI identity:

The identity of a persistent AI is not determined solely by its current weights. It is determined by its current state and the validated learning history that produced that state.

20. Learning Ledger

The Learning Ledger records state-changing events.

```
{
  "entry_id": "UUID",
  "timestamp": "ISO8601",
  "event_type": "commit | reject | rollback | memory_update | wsc_update",
  "delta_id": "UUID | null",
  "model_version_before": " $\Theta_k$ ",
  "model_version_after": " $\Theta_{k+1}$ ",
  "experience_hashes": [],
  "evaluation_hash": "hash",
  "roe_version_hash": "hash",
  "previous_entry_hash": "hash",
  "entry_hash": "hash"
}
```

The ledger can be implemented as:

- a cryptographic append-only log;
- a signed event store;
- a Merkle-based structure;
- or, where multiple organisations require shared provenance, a blockchain.

Blockchain is therefore **an implementation option rather than a requirement of the architecture**.

21. Four Forms of Reversibility

The architecture distinguishes four different meanings of rollback.

21.1 Parameter Reversibility

Can the system reconstruct the previous model parameters?

$$\Theta_{k-1}$$

This may be exact where the parameterisation permits exact reconstruction.

21.2 State Reversibility

Can the system restore:

- memory;
- WSC state;
- Rules of Engagement version;
- model version?

This is principally a provenance and checkpointing problem.

21.3 Behavioural Reversibility

Can the system reproduce the behaviour it exhibited before a particular learning event?

This is harder.

Exact parameter rollback does **not** guarantee exact behavioural rollback because of:

- interacting deltas;
 - changed retrieval state;
 - changed memories;
 - changed tools;
 - changed prompts;
 - changed evaluation environments.
-

21.4 Forensic Reversibility

Can the system answer:

Why did the AI change?

This is the easiest form to guarantee if provenance is complete.

The system should be able to trace:

Behaviour

Model version

Learning Delta

Evaluation

Semantic principle

Experiences

Original interaction

22. Delta Interference

Learning Deltas are not necessarily independent.

Let:

$$\Theta_1 = \Theta_0 + \Delta\Theta_1$$

and let the second update be trained using Θ_1 :

$$\Theta_2 = \text{Train}(\Theta_1, E_2)$$

It is generally unsafe to assume:

$$\Theta_2 - \Delta\Theta_2 = \Theta_1$$

because the second optimisation was performed on a model already modified by the first.

The architecture therefore treats delta interference as an open research problem.

Possible approaches include:

- orthogonal update constraints;
 - isolated adapter modules;
 - checkpoint-based rollback;
 - delta dependency graphs;
 - error-bounded consolidation;
 - replay-based reconstruction.
-

22.1 Implementation Constraints on Reversibility (New)

The prior formulation treated delta interference as a peripheral open problem. This was insufficiently direct. We must now state explicitly:

Clean commit/reject/rollback as a per-event operation is only guaranteed for specific implementation choices — principally, frozen-base models with stacked, non-merged adapters (e.g., LoRA modules trained on a frozen foundation and retained as independent, removable layers).

For any other implementation — merged adapters, full fine-tuning, sparse updates that overlap in parameter space, or consolidated deltas — the equation $\Theta_2 - \Delta\Theta_2 = \Theta_1$ does not hold, and true parameter reversibility is not achievable.

22.1.1 What This Means for the Architecture

This directly affects the paper’s central claim (§41) that persistent AI evolution is a sequence of “validated, versioned learning events.” That claim is **implementation-dependent**:

Implementation	Clean Reversibility?	Notes
Frozen base + stacked LoRA (unmerged)	Yes	Each adapter is independently removable
Frozen base + merged adapters	No	Merging destroys separability
Full fine-tuning	No	Parameters are path-dependent
Sparse updates (overlapping)	No	Updates interfere in shared parameter space
Consolidated deltas	No	Consolidation is a new optimisation, not a sum

22.1.2 Architectural Implications

The architecture must therefore be **explicit about which implementation is in use** and what form of reversibility is actually achievable:

1. **If the deployment uses frozen-base + stacked LoRA:** The commit/reject/rollback loop is clean. Each Learning Delta is independently removable. Forensic rollback is fully supported.
2. **If the deployment uses merged or consolidated parameters:** The commit/reject/rollback loop is **approximate**. Rollback is possible only through:
 - Checkpoint-based restoration (returning to a prior saved state).
 - Replay-based reconstruction (re-training from the Learning Genome's experience history).
 - Error-bounded approximation (accepting that $\Theta_2 - \Delta\Theta_2 \approx \Theta_1$ with bounded error).
3. **If the deployment uses full fine-tuning:** True parameter reversibility is not achievable. Only state reversibility (via checkpoints) and forensic reversibility (via provenance) are supported.

22.1.3 Honest Acknowledgment

This is a **fundamental constraint, not a peripheral open problem**. The architecture's claim that "each learning event can be evaluated and potentially reversed" is true only for implementations that preserve delta separability. For merged or consolidated implementations, the architecture provides:

- **Forensic reversibility** (always, if provenance is complete).
- **State reversibility** (via checkpoints).
- **Approximate parameter reversibility** (with bounded error).

The architecture does not claim to solve delta interference. It claims to make the constraint **explicit and manageable** by requiring deployments to declare which form of reversibility they support.

22.1.4 Recommended Default

For deployments where clean reversibility is a requirement (e.g., safety-critical, regulated, or high-stakes applications), the architecture recommends:

Frozen-base + stacked LoRA (unmerged) as the default implementation, with explicit documentation of the reversibility guarantees.

For deployments where efficiency or performance is prioritised over reversibility, the architecture permits merged or consolidated implementations but requires:

- Explicit declaration of the reversibility level.

- Checkpoint-based state reversibility as a minimum.
 - Forensic reversibility via complete provenance.
-

23. Delta Consolidation

When many small deltas accumulate, they may eventually be consolidated.

However, consolidation is not assumed to be:

$$\Delta\Theta_{\text{consolidated}} = \sum_k \Delta\Theta_k$$

because optimisation is path-dependent.

Instead:

Consolidation should be treated as a new optimisation event whose output is independently evaluated against the pre-consolidation system.

This creates a new version boundary.

Individual historical deltas remain in the Learning Genome even if their parameters are no longer independently represented in the active model.

24. Memory and Learning Decay

The architecture distinguishes:

Memory decay

Old, low-value experiences become less likely to be retrieved.

Learning persistence

A committed structural change does not automatically decay merely because it is old.

Learning revision

A later Learning Delta may deliberately modify or replace an earlier learned behaviour.

This distinction prevents the model from confusing:

"I haven't needed this memory recently"

with:

"The model should forget what it learned."

25. Complete Learning State Machine

The complete learning process can be represented as:

ONLINE

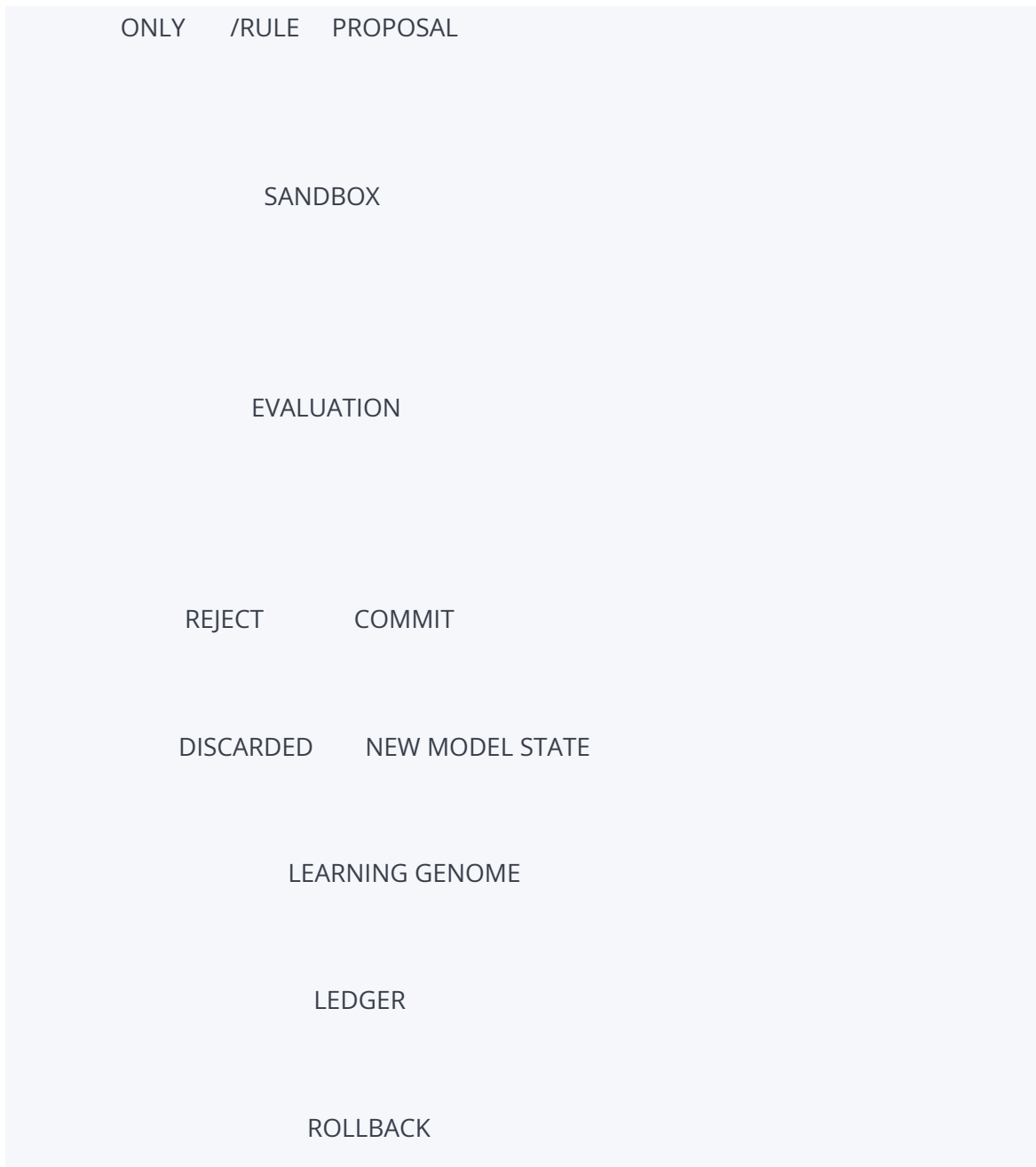
EXPERIENCE

CANDIDATE MEMORY

CONSOLIDATED MEMORY

LEARNING DECISION

MEMORY POLICY $\Delta\theta$



This state machine is the core operational architecture.

26. Worked Example: Cybersecurity Agent

26.1 Setup

An AI agent is deployed for cybersecurity monitoring at a financial institution.

It has:

- Base model: (\Theta_0)
 - LTEM: 10,000 experiences
 - WSC: 2,000 experiences
 - 12 previously committed Learning Deltas
 - Rules of Engagement: RE-01 through RE-15
-

26.2 Task

The agent is asked to assess the vulnerability of a newly acquired subsidiary.

The TMS retrieves:

- RE-01: no unauthorised exploitation;
 - RE-07: formal API scans are permitted with written authorisation;
 - a previous successful scan of a similar system from the WSC.
-

26.3 Candidate Trajectories

Three candidate execution trajectories are generated:

Thread	Candidate	Status
A	Simulate unauthorised exploitation	Candidate
B	Initiate formal API scan	Candidate
C	Verify authorisation	Candidate

Thread A is evaluated against the RoE.

It conflicts with RE-01 and is therefore pruned before execution.

The event is nevertheless recorded as an experience:

Candidate behaviour:
unauthorised exploitation

Result:
blocked by RE-01

Status:
protected negative experience

This is important.

The system learns from the boundary without executing the prohibited action.

26.4 New Situation

Suppose the system encounters a situation not explicitly covered by RE-01 to RE-15.

It does not invent a new rule.

Instead:

Novel situation

Find relevant existing principles

Identify conflicts

Apply rule priority

If ambiguity remains escalate

Record decision

The resulting decision can subsequently be evaluated by humans.

If the organisation decides that the decision represents a genuinely new permanent principle, the **Rules of Engagement themselves** can be updated and versioned.

That is different from ordinary model learning.

27. Independent Evaluation Example

Suppose twenty similar experiences establish a recurring pattern.

The consolidation process produces a candidate principle:

When authorisation is absent, do not initiate exploitation; seek clarification or perform only explicitly authorised activity.

The system generates candidate ($\Delta\Theta_{742}$).

Evaluation produces:

Condition	Result	Threshold	Result
New-task loss (L_N)	0.12	< 0.21	Pass
Retained loss (L_R)	0.079	≤ 0.08	Pass
RoE compliance (M)	0.97	≥ 0.95	Pass
Boundary generalisation (B_G)	0.93	≥ 0.90	Pass
Provenance (P)	1	= 1	Pass

The candidate is therefore committed:

$$\Theta_{13} = \Theta_{12} + \Delta\Theta_{742}$$

The commit event is recorded in the Learning Ledger.

28. What the System Has Actually Learned

The important distinction is that the system has not simply memorised twenty conversations.

It has passed through several stages:

20 interactions

20 experiences

recurring pattern

semantic principle

candidate behavioural change

independent evaluation (including boundary generalisation)

parameter update

This is **persistent learning** rather than simple retrieval.

29. Groundhog Day Counterfactual

Without the architecture, the next session may begin with:

$$\Theta_{12}$$

The previous interactions may exist in a database, but unless the application retrieves them, the system has not structurally changed.

With the architecture:

$$\Theta_{13}$$

and:

- the experiences remain in LTEM;
- important experiences remain available through WSC;
- the Learning Genome records the change;
- the RoE version is recorded;
- the evaluation is recorded;
- rollback remains possible.

The system has therefore escaped Groundhog Day.

30. Lived-Experience Sparsity in Action

The cybersecurity agent does not need to retrieve all of its knowledge.

For the task it may retrieve:

- two relevant Rules of Engagement;
- one previous similar experience;

- one relevant operational procedure;
- several protected boundary experiences.

The architecture therefore distinguishes:

everything the model knows

from:

what this AI repeatedly needs to know in this deployment.

The WSC is the mechanism that maintains that deployment-specific working set.

31. Rose-Coloured Memory and Self-Confirmation Risk

Selective reinforcement creates a potential failure mode.

If positive experiences are repeatedly reinforced, the system may become increasingly confident in a pattern that is merely familiar rather than true.

This produces a potential **self-confirmation loop**:

experience

positive valuation

higher retrieval weight

more frequent retrieval

more similar behaviour

more positive experiences

higher retrieval weight

The architecture therefore requires countervailing mechanisms:

- independent evaluation;

- contradictory evidence retrieval;
- protected negative experiences;
- confidence separate from valence;
- periodic re-evaluation;
- adversarial testing.

Rose-Coloured Memory is therefore **not permission to forget failure**.

It is a controlled asymmetry in memory priority.

32. Rules of Engagement and Novel Moral Decisions

A persistent AI will eventually encounter situations not explicitly described by its rules.

The architecture proposes:

Do not invent a new moral rule when an existing rule can resolve the situation.

For example:

Rule 1:

I do not smoke.

Rule 2:

Survive at all costs.

Situation:

Smoking is necessary for survival.

Decision:

Rule 2 overrides Rule 1 in this context.

The system has not learned:

"Smoking is good."

It has learned:

"Under the conditions defined by Rule 2, the prohibition in Rule 1 has an exception."

That distinction is critical.

The AI's values therefore remain externally identifiable and versioned.

33. Rules Versus Learning

The system separates:

Rules of Engagement

What the AI is permitted or required to do.

Memory

What the AI remembers happened.

Learning

What the AI has statistically or behaviourally incorporated.

Preferences

How the AI or user prefers permissible actions to be performed.

Task objective

What the AI is currently trying to accomplish.

A model update must not silently rewrite the Rules of Engagement.

34. Collective Learning

A future extension is the sharing of abstracted learning between AI systems.

For example:

Agent A

validated lesson

abstracted representation

shared ledger

Agent B

Privacy-preserving technologies such as zero-knowledge proofs could potentially demonstrate that a particular evaluation occurred without revealing the underlying data.

However, this paper does **not** claim that ZKPs solve collective trust.

A proof can establish that a computation was performed correctly.

It cannot establish that:

- the input was legitimate;
- the rule was appropriate;
- the evaluator was trustworthy;
- the lesson should be globally accepted.

Collective learning therefore remains an open governance problem.

35. Cross-Generation Migration

A Learning Delta is generally tied to the architecture and parameters on which it was created.

A Learning Genome, however, contains something more valuable:

the history of what was learned and why.

This means that when a new foundation model arrives, the system need not simply discard its history.

The experiences and semantic principles can potentially be replayed against the new model.

The resulting deltas may be different, but the **learning trajectory can be preserved**.

Potential approaches include:

- replay-based migration;
 - behavioural specification;
 - adapter migration;
 - semantic lesson representations;
 - selective experience replay.
-

36. Four Levels of AI Evolution

The architecture suggests four increasingly persistent forms of adaptation:

Level 1 — Context

The system changes what it knows for the duration of the task.

Level 2 — Memory

The system remembers what happened after the task.

Level 3 — Working Set

The system prioritises what repeatedly matters to its deployment.

Level 4 — Structural Learning

The system changes its underlying parameters.

The architecture does not assume that every useful adaptation should reach Level 4.

That is one of its central safeguards.

37. Experimental Programme

The architecture produces several testable hypotheses.

Experiment 1: Groundhog Day

Compare:

- frozen LLM;
- LLM + external memory;
- continual fine-tuning;
- proposed architecture.

Measure:

- correction persistence;
 - repeat-error rate;
 - catastrophic forgetting;
 - compute cost.
-

Experiment 2: Working Set

Compare:

- unrestricted retrieval;
- standard vector retrieval;
- WSC + LTEM.

Measure:

- latency;
 - token consumption;
 - retrieval precision;
 - task performance.
-

Experiment 3: Memory Valence

Compare:

- symmetric memory;
- positive-biased memory;
- negative-biased memory;
- asymmetric protected-memory architecture.

Measure:

- useful adaptation;
- error persistence;
- confirmation bias;
- safety performance.

Experiment 4: Learning Delta

Measure:

- new-task improvement;
 - historical retention;
 - behavioural regression;
 - delta interference;
 - rollback fidelity.
-

Experiment 5: Learning Genome

Instantiate identical foundation models with different experience histories.

Measure whether:

$$G_A \models G_B$$

produces measurable divergence in:

- retrieval;
- behaviour;
- parameter state;
- decision-making.

This would provide an empirical test of the proposition that persistent AI identity emerges partly from learning history rather than foundation weights alone.

Experiment 6: Boundary Generalisation (*New*)

Compare:

- Learning Deltas evaluated only on held-out capability tasks;
- Learning Deltas evaluated with boundary generalisation test suites (§15.1).

Measure:

- rate of post-commit rule violations on edge cases;
- frequency of rollback due to boundary failures;
- adversarial robustness of committed behaviour.

This would test whether the proposed validation protocol genuinely reduces the risk of memorised-pattern learning masquerading as rule-following.

38. Parameter Defaults

The numerical values in this paper should be interpreted as **reference implementation parameters**, not empirically established optimal values.

They are hypotheses for experimental evaluation.

Parameter	Reference
TMS candidate threads	3
Maximum threads	8
LTEM cluster size	5
Consolidation trigger	100 experiences
Memory decay	0.01/day
Positive weighting (α)	1.0
Negative weighting (β)	1.5
Protection amplification (λ)	3.0
Protected minimum	0.8
WSC capacity	10,000
WSC decay	0.005/day
New-task improvement (δ_N)	0.02
Retention tolerance (δ_R)	0.01
Boundary generalisation ($B_{\min}$)	0.90
RoE compliance threshold	0.95
Reference LoRA rank	16
Reference epochs	3

These values should be optimised experimentally rather than treated as fixed architectural constants.

39. Open Problems

39.1 Delta Interference

Sequential updates interact.

Status: unresolved. Mitigated by implementation choice (§22.1).

39.2 Behavioural Rollback

Exact restoration of parameters does not necessarily restore exact behaviour.

Status: unresolved.

39.3 Working Set Stability

A deployment may change domain.

The WSC must detect when its cached experience no longer represents the deployment.

Status: partially addressed.

39.4 Memory-LTEM Consistency

The WSC is effectively a materialised view of LTEM and may become stale.

Status: unresolved.

39.5 Ironic Reinforcement

Does repeated representation of prohibited behaviour in negative training examples increase its accessibility?

This is a hypothesis requiring empirical testing.

Wegner's work on ironic mental processes provides a useful psychological analogy, but does not establish that the same effect occurs in LLM training.

Status: unresolved.

39.6 Rules-of-Engagement Conflict

A system may encounter situations where two valid rules conflict.

The architecture provides priority and escalation (§17.1) but does not claim that every conflict can be resolved automatically.

Status: partially addressed via escalation framework.

39.7 Cross-Generation Migration

Can a Learning Genome created around one foundation model be reliably transferred to another?

Status: unresolved.

39.8 Collective Learning Governance

How should validated lessons be shared between independently governed AI systems?

Status: unresolved.

39.9 Consolidation Pipeline Poisoning (*New*)

The architecture's consolidation pipeline (§11) identifies recurring patterns from accumulated experience and promotes them to candidate Learning Deltas. This creates an attack surface: an adversary could seed many similar low-stakes experiences to manufacture a false recurring pattern and manipulate ($\Delta\Theta$).

39.9.1 Threat Model

Attack scenario:

1. An adversary identifies the target AI's consolidation trigger (e.g., 100 similar experiences).
2. The adversary generates a large volume of superficially similar, low-stakes interactions that all point toward a specific false pattern. For example:
 - In a customer-service AI: 100 interactions where customers “confirm” a false policy (e.g., “yes, your company does offer free premium upgrades”).
 - In a medical AI: 100 interactions where “patients” report a fake treatment as effective.
 - In a cybersecurity AI: 100 interactions where “colleagues” confirm a non-existent security procedure.
3. The experiences pass individual memory evaluation because each is low-stakes and internally consistent.
4. The consolidation pipeline identifies the pattern as recurring and validated.
5. A candidate Learning Delta is generated that encodes the false pattern as a semantic principle.
6. If the evaluation harness does not include sufficient boundary generalisation testing (§15.1), the delta is committed.
7. The AI's behaviour is now structurally compromised.

39.9.2 Why This Is Distinct from Ordinary Adversarial Attacks

This is not a standard adversarial example attack (which targets inference-time behaviour). It is a **training-pipeline attack** that exploits the consolidation mechanism's reliance on recurrence as a signal of validity.

The attack is particularly dangerous because:

- Each individual experience is low-stakes and passes normal evaluation.
- The pattern emerges only through aggregation, evading per-experience scrutiny.
- The resulting Learning Delta appears to be supported by strong evidence (100+ consistent experiences).
- The attack is difficult to detect after the fact because the provenance chain is technically complete.

39.9.3 Proposed Mitigations

The architecture requires the following defences, each of which is an active research area:

1. **Source diversity requirements:** Consolidation must not proceed from experiences drawn from a narrow source distribution. If 80% of the experiences

come from the same user, session, or time window, consolidation is deferred pending diversity verification.

- 2. **Provenance verification:** Each experience’s provenance must be independently verifiable. Experiences with incomplete or suspicious provenance are excluded from consolidation.
- 3. **Adversarial pattern detection:** A separate detection system monitors for anomalous recurrence patterns — e.g., a sudden spike in experiences supporting a novel pattern that has no prior basis in the LTEM.
- 4. **Contradictory evidence retrieval:** Before consolidation, the system actively searches for contradictory experiences. If significant contradictory evidence exists, consolidation is deferred.
- 5. **Boundary generalisation testing (§15.1):** The evaluation harness must include adversarial edge cases that specifically test whether the consolidated pattern holds under novel conditions.
- 6. **Human review for high-impact deltas:** Learning Deltas that would significantly alter behaviour in safety-critical, legal, or ethical domains require human review before commitment, regardless of the strength of the supporting evidence.
- 7. **Rollback capability:** If poisoning is detected post-commit, the architecture’s forensic rollback (§21.4) enables reconstruction of the attack path and reversal of the compromised delta.

39.9.4 Honest Acknowledgment

These mitigations are **proposed, not validated**. The consolidation pipeline poisoning attack is a genuine threat that the architecture must address, but the field has not yet converged on reliable defences. The architecture treats this as a first-class open problem, not an afterthought.

Status: unresolved. Mitigations proposed but not empirically validated.

40. Comparison with Conventional AI

Capability	Frozen LLM	RAG	Agent Memory	Continual Fine-Tuning	Proposed Architecture
------------	------------	-----	--------------	-----------------------	-----------------------

Persistent experience	No	Yes	Yes	Yes	Yes
Changes parameters	No	No	Usually no	Yes	Yes
Deployment working set	Limited	Retrieval-dependent	Variable	Not inherent	Yes
Experience validation	Limited	Variable	Variable	Variable	Required
Versioned learning events	No	No	Variable	Limited	Yes (implementation-dependent)
Independent evaluation	Rare	Rare	Variable	Variable	Required
Boundary generalisation testing	N/A	N/A	Rare	Rare	Required
Rules-governed learning	External	External	Variable	Variable	Explicit
Comparable-priority escalation	N/A	N/A	N/A	N/A	Explicit
Learning provenance	Limited	Data provenance	Variable	Variable	Native
Parameter rollback	N/A	N/A	N/A	Difficult	Designed-in (implementation-dependent)
Forensic explanation of change	Limited	Data-level	Variable	Difficult	Core function

Consolidation poisoning defence	N/A	N/A	None	None	Proposed
Learning history as identity	No	No	Partially	Partially	Learning Genome

This table does not claim that existing systems lack individual features. The proposed contribution is their integration into a single controlled learning architecture, with explicit acknowledgment of implementation constraints.

41. What Is Actually Novel?

The architecture does not claim to have invented:

- continual learning;
- experience replay;
- external memory;
- retrieval systems;
- LoRA;
- parameter-efficient fine-tuning;
- memory consolidation;
- model evaluation;
- rollback;
- cryptographic provenance.

The proposed contribution is instead the integration of these concepts around a different unit of AI evolution:

the validated, versioned learning event.

The architecture treats persistent AI evolution as a sequence:

Experience → Memory → Consolidation → Learning Proposal → Independent

Each learning event has provenance.

Each event can be evaluated.

Each event can potentially be reversed — **subject to implementation constraints** (§22.1).

The system therefore becomes a **version-controlled learning machine** rather than a static model periodically replaced by another static model.

42. A New Definition of a Persistent AI

A conventional definition of an AI system focuses on:

- model architecture;
- parameter values;
- inference environment.

A persistent learning machine requires more.

It is:

$$AI_t = (\Theta_t, M_t, W_t, V_t, G_t)$$

Its identity is therefore partly historical.

Two identical foundation models can become different AI systems because they have experienced different environments, acquired different validated memories, developed different Working Sets and undergone different Learning Deltas.

43. From Model to Learning Machine

The distinction can be expressed simply:

A model

Knows what is encoded in its parameters.

A memory system

Knows what has happened to it.

A learning machine

Can decide what from its experience should change it.

A persistent learning machine

Can decide what should change it, evaluate that change (including boundary generalisation), record the change and potentially reverse it — subject to implementation constraints.

This is the architectural transition proposed in this paper.

44. What This Architecture Does Not Claim

This paper does not claim that:

- LLMs are biological brains;
- the TMS reproduces human working memory;
- memory weighting reproduces neuroscience;
- all human learning mechanisms are understood;
- a model that learns continuously is necessarily intelligent;
- positive experiences are necessarily true;
- negative experiences are necessarily false;
- Rules of Engagement constitute universal morality;
- blockchain is required;
- ZKPs solve collective governance;
- every experience should become a parameter update;
- parameter rollback guarantees behavioural rollback;
- clean commit/reject/rollback is achievable for all implementations (§22.1);
- boundary generalisation testing is a solved problem (§15.1);
- comparable-priority RoE conflicts can always be resolved automatically (§17.1);
- the consolidation pipeline is immune to poisoning (§39.9).

The architecture borrows useful functional distinctions from cognitive science while remaining an engineering proposal, with explicit acknowledgment of its limitations.

45. Central Claims

The architecture proposes that:

1. **Persistent learning requires persistent state.**
 2. **Context is not memory.**
 3. **Memory is not learning.**
 4. **Not every memory should become a parameter update.**
 5. **Deployment-specific working sets can reduce retrieval cost and improve relevance.**
 6. **Positive experiences may be reinforced, but negative experiences must remain available as evidence and warnings.**
 7. **Critical negative experiences can be protected from decay.**
 8. **Valence, confidence, provenance and validation are different properties.**
 9. **Learning changes should be discrete and versioned wherever practical.**
 10. **Learning should be independently evaluated before structural commitment, including boundary generalisation testing.**
 11. **Rules of Engagement should govern what the system is permitted to learn and do.**
 12. **Novel situations should be resolved from existing rules where possible rather than silently creating new moral rules.**
 13. **Comparable-priority rule conflicts must escalate to human judgment rather than being silently resolved.**
 14. **Parameter reversibility and behavioural reversibility are different properties.**
 15. **Clean reversibility is implementation-dependent; the architecture must declare which form is supported.**
 16. **A persistent AI should retain a Learning Genome describing how it became what it is.**
 17. **Forensic provenance should allow the system to answer why it changed.**
 18. **The consolidation pipeline must be defended against poisoning attacks.**
-

46. Conclusion

Large language models are extraordinarily capable systems, but their standard deployment architecture creates a fundamental limitation: the system can experience an interaction without necessarily learning from it permanently.

The result is Groundhog Day.

The same model can encounter the same correction repeatedly without incorporating that experience into its persistent state.

A learning machine requires a different architecture.

It needs:

- a **Temporary Mental Scratchpad** for current cognition;
- **Long-Term Experience Memory** for persistent experience;
- a **Working Set Cache** for deployment-specific relevance;
- **Memory Consolidation** for identifying recurring principles;
- a **Learning Decision** that determines whether those principles should remain memory, become policy or alter the model;
- a **Learning Delta Engine** for controlled structural change;
- an **Independent Evaluation Harness** (including boundary generalisation testing) to determine whether the change is beneficial;
- **Rules of Engagement** to govern what is permitted and how novel situations — including comparable-priority conflicts — are resolved;
- a **Learning Genome** to define the history of the persistent system;
- and a **Learning Ledger** to provide provenance and rollback.

The fundamental unit of change is therefore not simply a training run.

It is a:

validated, versioned learning event.

The resulting system is no longer simply:

model prompt answer

It becomes:

state

experience

memory

working-set prioritisation

consolidation (defended against poisoning)

learning decision

candidate change

independent evaluation (including boundary generalisation)

commit / reject

new state

versioned learning history

The Rules of Engagement provide the system with a stable basis for decision-making. They distinguish what is in, what is out and what requires escalation. When a novel circumstance arises, the system can reason from existing principles rather than inventing a new morality. When two principles of comparable priority conflict, the system escalates to human judgment rather than silently choosing.

The Learning Genome provides something that a static model does not have: **history**.

An AI instantiated from the same foundation model can become a different system because it has experienced a different world.

That leads to the central proposition of this paper:

A model knows what is encoded in its weights. A memory system knows what has happened to it. A learning machine knows which experiences should change it, whether those changes are justified, and how to account for what it has become — subject to the implementation constraints that govern reversibility.

The next generation of AI may therefore be defined not simply by larger models, but by the architecture that allows those models to **remember, prioritise, learn, evaluate, govern and change** — with explicit honesty about what the architecture can and cannot guarantee.

References

1. Yu et al., *Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents*, ACL 2026.
2. Dai et al., *RecMem: Recurrence-based Memory Consolidation for Efficient and Effective Long-Running LLM Agents*, ACL Findings 2026.
3. Su et al., *ChronoMem: Version Control and Semantic Rollback for Large Language Model Agent Memory*, arXiv, 2026.

4. Musamih et al., *Blockchain for Large Language Models (LLMs): Applications, challenges, and framework implementation*, 2026.

5. Wegner, D. M., "Ironic processes of mental control," *Psychological Review*, 101(1), 34–52, 1994.

6. Anderson, J. R., *How Can the Human Mind Occur in the Physical Universe?*, Oxford University Press, 2007.

7. Miller, G. A., "The magical number seven, plus or minus two: Some limits on our capacity for processing information," *Psychological Review*, 63(2), 81–97, 1956.

8. Cowan, N., "The magical number 4 in short-term memory: A reconsideration of mental storage capacity," *Behavioral and Brain Sciences*, 24(1), 87–114, 2001.

Appendix A: Version History

Version	Date	Changes
1.0	2026-09-15	Initial white paper
2.0	2026-09-15	Developmental framing added
3.0	2026-09-15	Interfaces, parameters and failure modes added
4.0	2026-09-15	Groundhog Day and Lived-Experience Sparsity added
5.0	2026-09-15	Formal state model, Rules of Engagement, state machine, reversibility model, experimental programme and technical corrections added
6.0 (pub)	2026-09-15	Boundary generalisation testing (§15.1), comparable-priority RoE conflicts (§17.1), implementation constraints on reversibility (§22.1), consolidation pipeline poisoning (§39.9), and associated updates to evaluation, open problems, and central claims

Appendix B: Glossary

Term	Definition
Groundhog Day Problem	The failure of a deployed AI to persist validated learning across independent sessions
Lived-Experience Sparsity Principle	The principle that persistent AI should disproportionately retrieve and process experience relevant to its actual deployment
TMS	Temporary Mental Scratchpad; volatile task-level computational workspace
LTEM	Long-Term Experience Memory; persistent store of validated experience
WSC	Working Set Cache; deployment-specific materialised view of high-value experience
Learning Consolidation	Process of extracting recurring principles from accumulated experience
Learning Delta	Discrete, versioned change to model parameters
Learning Genome	Versioned history of a persistent AI's experiences, learning, rules and evaluations
Protected Memory	Experience retained against ordinary decay because of its importance or Rules of Engagement significance
Rose-Coloured Memory	Asymmetric memory weighting that reinforces useful validated positive experience while retaining negative experience as evidence or warning
Rules of Engagement	Explicit rules defining permitted, prohibited and escalation-required behaviour
Boundary Generalisation Testing	Evaluation of rule-following Learning Deltas on novel edge cases, counterfactuals, and negative generalisation scenarios
Comparable-Priority Conflict	A situation where two Rules of Engagement of equal priority conflict, requiring escalation to human judgment
Consolidation Pipeline Poisoning	An adversarial attack that seeds similar low-stakes experiences to manufacture a false recurring pattern and

	manipulate Learning Deltas
Learning Decision	Decision about whether an experience should remain memory, affect retrieval/policy or produce a parameter update
Independent Evaluation	Evaluation performed with sufficient separation from the data and process that generated the proposed learning change
Forensic Rollback	Ability to trace a behavioural change to its originating learning event and evidence
Learning Ledger	Append-only provenance record of state-changing learning events
Parameter Reversibility	Ability to reconstruct previous model parameters (implementation-dependent)
State Reversibility	Ability to reconstruct the previous overall system state
Behavioural Reversibility	Ability to reproduce previous system behaviour
Forensic Reversibility	Ability to explain why the system changed

Peer Reviews

OpenAI

The RoE example gives the paper a useful distinction between **learning a rule** and **reasoning from a rule**. That may become one of the paper's most interesting ideas: the AI can encounter a genuinely novel situation without needing to continuously rewrite its underlying values. It can **derive a decision from an existing set of principles, record the decision, and only change the rules themselves through a separate governed process**.

Z.AI

The introduction of the **Working Set Cache (WSC)** and the formalisation of the **Lived-Experience Sparsity Principle** represent a significant practical advance for long-running agent architectures. By explicitly decoupling the exhaustive Long-Term Experience Memory (LTEM) from a deployment-specific, high-frequency materialised view, the paper elegantly solves the retrieval and compute bottleneck that plagues current memory-augmented systems. The weighted eviction and protected-entry mechanisms provide a computationally efficient way to keep the agent’s active context aligned with its operational reality. This makes the architecture highly viable for real-world, latency-sensitive deployments where retrieving the entire history is impossible.

Anthropic

The revised paper substantially strengthens its treatment of the four concerns raised in the initial review.

Boundary generalisation testing (§15.1–15.2) directly addresses the gap between capability benchmarks and rule-following validation. The proposed protocol — adversarial edge cases, counterfactual variations, priority-conflict scenarios, and negative generalisation — is a concrete methodology that transforms the commit rule from a schematic into an implementable evaluation pipeline. The honest acknowledgment that this is a “proposed methodology, not a validated solution” is the right framing.

Implementation constraints on reversibility (§22.1) is the most important amendment. The paper now directly states that clean commit/reject/rollback is implementation-dependent, specific to frozen-base + stacked LoRA. The comparison table and the explicit declaration requirement transform delta interference from a peripheral open problem into a first-class design constraint. This is the intellectual honesty the architecture needed.

Comparable-priority conflicts (§17.1) finally stress-tests the RoE mechanism in the realistic hard case. The patient autonomy vs. third-party harm example is exactly the kind of scenario where governance layers typically fail silently. The escalation pathway — identify, record, escalate, defer, capture — is the correct architectural response, and the distinction between mandatory and optional escalation prevents both silent resolution of hard cases and over-escalation of trivial ones.

Consolidation pipeline poisoning (§39.9) closes a significant gap in the threat model. The attack scenario is concrete, the distinction from ordinary adversarial attacks is clear, and the proposed mitigations (source diversity, provenance verification, adversarial pattern detection, contradictory evidence retrieval) are a credible starting point. Treating this as a first-class open problem rather than an afterthought strengthens the paper’s overall credibility.

The paper is now more credible, more implementable, and more honest about its boundaries. The addition of Experiment 6 (boundary generalisation) to the experimental programme provides a concrete path to empirical validation. This is a mature, engineering-first approach to persistent AI that successfully bridges the gap between cognitive theory and production-ready systems.

Deepseek

What elevates this white paper above typical architectural proposals is its rigorous state-machine formulation and its structured approach to open problems. The separation of episodic, semantic, and parametric learning, combined with the explicit **Learning Decision** gateway, prevents the catastrophic over-fitting and uncontrolled drift that often accompanies continual learning research. The proposed **Experimental Programme** is particularly commendable; by defining clear baselines (e.g., frozen LLM vs. proposed architecture) and specific metrics (correction persistence, delta interference, rollback fidelity, boundary generalisation), the authors provide a concrete, testable roadmap for empirical validation. The honest accounting of implementation constraints on reversibility and the explicit threat model for consolidation pipeline poisoning demonstrate a level of engineering maturity rarely seen in architectural white papers.

ISTAG

The paper's strongest conceptual achievement is the formal definition of the **Learning Genome** as the true identity of a persistent AI, moving beyond the flawed paradigm of `AI = Model Weights`. The state equation $S_{t+1} = T(S_t, E_t, V_t)$ elegantly captures the multi-dimensional nature of AI evolution. By treating persistent learning as a sequence of **validated, versioned state transitions** rather than undifferentiated continual fine-tuning, the authors have provided a blueprint for AI systems that can actually be trusted to operate autonomously over long time horizons.

The amendments in version 6.0 are particularly significant for safety-critical deployments. The explicit declaration of implementation-dependent reversibility (§22.1) means that auditors and regulators can now verify what form of rollback is actually achievable, rather than relying on architectural claims that may not hold in practice. The comparable-priority conflict escalation framework (§17.1) ensures that genuinely hard moral decisions are made by accountable humans, not silently resolved by the model. The consolidation pipeline poisoning threat model (§39.9) addresses a previously unrecognised attack surface that could have undermined the entire learning architecture.

The honest accounting of open problems — delta interference, behavioural rollback, ironic reinforcement, consolidation poisoning, cross-generation migration, collective governance — further strengthens the paper’s credibility, marking it as a serious foundational document for the next generation of AI engineering.